

计算机视觉第三次作业

20337025 崔璨明

计算机科学与技术（人工智能与大数据）

2023 年 6 月 15 日

目录

1 实验题目	2
1.1 半监督图像分类	2
1.2 扩散生成模型	2
2 实验原理	2
2.1 半监督图像分类	2
2.2 扩散生成模型	4
3 实现步骤与代码	4
3.1 半监督图像分类	4
3.1.1 MixMatch	4
3.1.2 FixMatch	6
3.2 扩散生成模型	7
4 实验结果	11
4.1 半监督图像分类	11
4.1.1 图像分类效果	11
4.1.2 和 TorchSSL 的对比	15
4.1.3 MixMatch 和 FixMatch 的相同点和不同点	15
4.2 扩散生成模型	15
5 实验总结	16
5.1 遇到的困难	16
5.2 实验感想	16

1 实验题目

1.1 半监督图像分类

- 实验内容:
 1. 基于 MixMatch 的 CIFAR-10 数据集半监督图像分类
 2. 基于 FixMatch 的 CIFAR-10 数据集半监督图像分类
- 实验要求:
 1. 阅读原始论文和相关参考资料, 基于 Pytorch 实现 MixMatch 和 FixMatch 半监督图像分类算法, 按照原始论文的设置, MixMatch 和 FixMatch 均使用 WideResNet-28-2 作为 Backbone 网络, 即深度为 28, 扩展因子为 2, 在 CIFAR-10 数据集上进行半监督图像分类实验, 报告算法在分别使用 40, 250, 4000 张标注数据的情况下的图像分类效果 (标注数据随机选取指定数量)
 2. 使用 TorchSSL 中提供的 MixMatch 和 FixMatch 的实现进行半监督训练和测试, 对比自己实现的算法和 TorchSSL 中的实现的效果
 3. 提交源代码, 并提交实验报告, 描述主要的实现步骤, 并分析对比 MixMatch 和 FixMatch 的相同点和不同点。

1.2 扩散生成模型

- 实验内容: 基于 DDPM 的 MNIST 和 Fashion-MNIST 图像生成
- 实验要求:
 1. 阅读原始论文和相关参考资料, 基于 Pytorch 实现简单版 DDPM (不要求改进), 并在 MNIST 和 Fashion MNIST 上进行训练, 测试生成效果
 2. 在 MNIST 和 Fashion MNIST 上完成 DDPM 的训练, 每个数据集展示 10 个生成的图片, 并以动图形式在 MNIST 和 Fashion MNIST 上各展示一个生成效果图。
 3. 提交源代码和实验报告, 对实现过程中的关键步骤进行必要说明。

2 实验原理

2.1 半监督图像分类

半监督学习的核心思想是利用无标记数据的信息来改进模型的学习效果。在半监督学习中, 使用少量标记数据进行有监督学习, 同时利用大量无标记数据的信息。通过充分利用无标记数据的潜在结构和分布特征, 半监督学习可以帮助模型更好地泛化和适应未标记数据。有许多经典的半监督图像分类算法, 如: Model[2], Mean Teacher[3], MixMatch[4], FixMatch[5] 等。

MixMatch [BCG⁺19] 采用两种方式利用无标注数据, 即熵最小化和一致性正则化。具体来说, MixMatch 首先对无标注数据进行 K 次增广, 并获得对应的 K 个输出, 并计算模型平均的预测分布, 之后, MixMatch 使用一个“sharpening”函数来对无标注数据的平均预测进行锐化, 并把锐化后的预测分布作为该无标注数据的标签。获得对无标注数据的标签后, 将标注数据和无标注数据结合起来, 通过 Mixup 的方式完成新数据集的构建, 最后使用新的数据集进行训练。

MixMatch 算法流程如图 1 所示:

1. 将无标注数据多次增强，过一次模型输出其预测。
2. 对于多个结果进行平均，sharpen 后输出伪标签 (sharpen 将概率分布分得更开)。
3. 将有标注数据和无标注做 mixup (将原来矩阵和 shuffle 后的矩阵相加)，伪标签作为无标注的 target. mixup 后的矩阵顺序仍然为 $[X_l, X_u1, ..., X_u2]$ 。
4. 再将 mixup 后的过一次模型，输出预测，对有标注数据求交叉熵损失，对无标注数据通过伪标签求 L_2 损失。



图 1: MixMatch 流程图

FixMatch 结合了伪标签和一致性正则化来实现对无标注数据的高效利用，训练过程包括两个部分，有监督训练和无监督训练。有标注的数据，执行有监督训练，和普通分类任务训练没有区别。没有标注的数据，首先将其经过弱增强之后送到模型中推理获取伪标签，然后再将其经过强增强送到模型中进行预测，并利用生成的伪标签监督模型对强增强数据的预测。模型对弱增强数据的预测，只有大于一定阈值条件时才执行伪标签的生成，并使用该伪标签来进行无标注图像的训练。

FixMatch [SBC+20] 算法让强增强的预测结果和弱增强的伪标签一致，采用了阈值来过滤掉置信度低的伪标签。相比 MixMatch 更为简单。算法流程如图 2 所示：

1. 输入 $[X_l, X_{u_{weak}}, X_{u_{strong}}]$ ，通过模型输出预测。
2. 将 weak 增强得到且大于阈值的伪标签转为 onehot 硬标签，与 strong 增强的计算交叉熵损失作为 L_u 。有标注的结果计算交叉熵损失作为 L_x 。

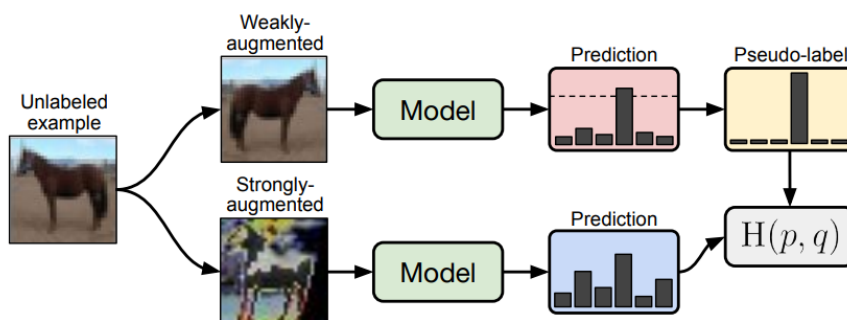


图 2: FixMatch 流程图

2.2 扩散生成模型

扩散模型 (Diffusion Model) [YZS+22] 是近年来提出的一种新的生成模型。生成模型是一种概率模型，我们希望学习一个简单的分布，来学习映射到一个我们想要生成的真实图像的分布，推理时，我们只需要从简单分布中取一个样本，比如高斯分布中的一个噪声样本，经过模型计算就能映射到真实图像类中的一个样本，即生成一张真实的图像。与之前的对抗生成式模型通过对抗隐式学习从噪声分布到真实图像分布不同，扩散模型基于马尔科夫链，将生成过程建模为一个加噪，去噪的过程。训练时，先对一张真实图像，不断往其中添加噪声使其变成标准高斯噪声（前向过程），然后训练网络学习前向过程中加噪声的逆过程，从标准高斯噪声中不断去噪，还原回原始数据（逆向过程）。学习完成后，就能够从已知的标准高斯分布中采样一个噪声数据，利用学到的去噪过程生成符合原始数据分布的新数据。

3 实现步骤与代码

3.1 半监督图像分类

MixMatch 和 FixMatch 的实现参考了这篇博客：<https://zhuanlan.zhihu.com/p/539210221>。

3.1.1 MixMatch

MixMatch 的实现见 code/mixmatch 文件夹，其中：

- cifar10.py：用于处理 CIFAR-10 数据集的数据预处理和数据加载。包括将数据集划分为带标签样本、无标签样本、验证样本和测试样本；以及一些增强操作：图像标准化、图像转置、图像填充、随机裁剪、随机翻转和添加高斯噪声等。
- wideresnet.py：定义了一个宽残差网络（Wide Residual Network）。其中 BasicBlock 类定义了基本的残差块。它包含两个卷积层和一个残差连接。在每个卷积层之间使用了批归一化和 LeakyReLU 激活函数。可以选择性地添加 dropout 进行正则化。NetworkBlock 类定义了网络块，由多个 BasicBlock 组成。它根据传入的参数构建了指定数量的 BasicBlock。WideResNet 类定义了整个宽残差网络。它包含了卷积层、多个网络块以及全局平均池化层和分类器。在初始化过程中，根据给定的深度、widen_factor 和 dropRate 确定各个层的通道数和网络块的数量。forward() 方法按照网络的结构依次调用了卷积层、网络块和全局平均池化层，并最终通过全连接层进行分类。
- eval.py：定义了一个 accuracy 函数用于计算给定输出和目标之间的预测准确度。
- misc.py：一些辅助函数，如 get_mean_and_std() 函数用于计算数据集的均值和标准差。init_params(net) 函数用于初始化网络层的参数。AverageMeter 类用于计算并存储平均值和当前值。
- train.py：实现了使用 MixMatch 算法进行半监督学习的训练过程。其中 train() 函数用于执行模型的训练过程。它接受 model、dataloader、optimizer、损失函数和当前轮数作为参数。将模型设置为训练模式，然后遍历每个批次，执行前向传播、计算损失、反向传播和参数更新的操作。此外，它还会更新 EMA 模型的参数，并返回模型。test() 函数用于模型的测试。函数会将模型设置为评估模式，然后遍历数据加载器中的每个批次，执行前向传播并计算准确率。最后，它会返回计算得到的准确率。main() 函数为主函数，用于进行训练和测试。在训练过

程中，使用 MixMatch 算法进行模型训练，并使用 EMA 模型计算参数的滑动平均值。训练过程中会记录训练信息，并根据特定的训练轮数保存模型状态和记录日志。

关键代码解析：

首先是通过水平翻转、裁剪实现图像的增强：

```
1 transform_train = transforms.Compose([
2     dataset.RandomPadandCrop(32), # 随机填充和裁剪
3     dataset.RandomFlip(), # 随机翻转
4     dataset.ToTensor(), # 转换为张量
5 ])
6 transform_val = transforms.Compose([
7     dataset.ToTensor(), # 转换为张量
8 ])
```

迭代过程如下，两个 try except 保证迭代器完全迭代一次后，重新加载迭代器，继续迭代，直到达到指定次数才跳转下一个 epoch：

```
1 for batch_idx in range(args.train_iteration):
2     try:
3         inputs_x, targets_x = labeled_train_iter.next() # 获取有标签训练集的输入和目标
4     except:
5         labeled_train_iter = iter(labeled_trainloader) # 若遍历完有标签训练集，则重新创建迭代器
6         inputs_x, targets_x = labeled_train_iter.next() # 获取有标签训练集的输入和目标
7     try:
8         (inputs_u, inputs_u2), _ = unlabeled_train_iter.next() # 获取无标签训练集的输入
9     except:
10        unlabeled_train_iter = iter(unlabeled_trainloader) # 若遍历完无标签训练集，则重新创建迭代器
11        (inputs_u, inputs_u2), _ = unlabeled_train_iter.next() # 获取无标签训练集的输入
```

按照论文中的设置, $K=2$, 所以进行两次扩增, 然后求输出概率的均值, output_u 和 output_u2 分别为两次扩增后的模型的输出结果：

```
1 outputs_u = model(inputs_u)
2 outputs_u2 = model(inputs_u2)
3 p = (torch.softmax(outputs_u, dim=1) + torch.softmax(outputs_u2, dim=1)) / 2 # 求平均值
```

计算 Sharpening 结果：

```
1 pt = p**(1/args.T)
2 targets_u = pt / pt.sum(dim=1, keepdim=True)
3 targets_u = targets_u.detach()
```

MixUp 的实现过程：

```
1 all_inputs = torch.cat([inputs_x, inputs_u, inputs_u2], dim=0) # 将有标签和无标签训练集的输入拼接在一起
2 all_targets = torch.cat([targets_x, targets_u, targets_u2], dim=0) # 将有标签和无标签训练集的目标拼接在一起
3 l = np.random.beta(args.alpha, args.alpha) # 采样一个随机数
4 l = max(l, 1 - l)
5 idx = torch.randperm(all_inputs.size(0)) # 生成一个随机的索引排列
6 input_a, input_b = all_inputs, all_inputs[idx] # 根据索引排列将输入数据划分为两个部分
7 target_a, target_b = all_targets, all_targets[idx] # 根据索引排列将目标数据划分为两个部分
8 mixed_input = l * input_a + (1 - l) * input_b # 混合输入数据
9 mixed_target = l * target_a + (1 - l) * target_b # 混合目标数据
```

最后计算损失函数并反向传播训练即可：

```

1 logits = [model(mixed_input[0])] # 对混合后的第一个输入进行模型前向传播得到 logits
2 for input in mixed_input[1:]: # 对剩余的混合输入逐个进行模型前向传播
3     logits.append(model(input))
4 # 将交错的样本重新排列
5 logits = interleave(logits, batch_size)
6 logits_x = logits[0] # 有标签样本的 logits
7 logits_u = torch.cat(logits[1:], dim=0) # 无标签样本的 logits
8 # 计算有标签样本的损失 Lx, 无标签样本的损失 Lu, 以及权重 w
9 Lx, Lu, w = criterion(logits_x, mixed_target[:batch_size], logits_u, mixed_target[batch_size:],
                        epoch + batch_idx / args.train_iteration)
10 loss = Lx + w * Lu # 总 loss

```

3.1.2 FixMatch

FixMatch 的实现见 code/fixmatch 文件夹, 其中:

- cifar10.py: 和 MixMatch 中的一样, 用于处理 CIFAR-10 数据集的数据预处理和数据加载。包括将数据集划分为带标签样本、无标签样本、验证样本和测试样本; 以及一些增强操作: 图像标准化、图像转置、图像填充、随机裁剪、随机翻转和添加高斯噪声等。
- misc.py: 一些辅助函数, 如 get_mean_and_std() 函数用于计算数据集的均值和标准差, 类 AverageMeter 用于计算和存储平均值和当前值。
- wideresnet.py: 同样, 定义了 WideResNet 模型的结构, 并提供了构建模型的辅助函数。
- ema.py: 实现了模型的指数移动平均。
- trian: 实现了使用 FixMatch 算法进行半监督学习的训练过程, train() 函数用于执行模型的训练, test() 函数用于模型的测试。

关键代码解析:

读取 batch, 进行迭代的过程:

```

1 for batch_idx in range(args.eval_step):
2     try:
3         inputs_x, targets_x = labeled_iter.next()
4     except:
5         if args.world_size > 1:
6             labeled_epoch += 1
7             labeled_trainloader.sampler.set_epoch(labeled_epoch)
8             labeled_iter = iter(labeled_trainloader)
9             inputs_x, targets_x = labeled_iter.next()
10    try:
11        (inputs_u_w, inputs_u_s), _ = unlabeled_iter.next()
12    except:
13        if args.world_size > 1:
14            unlabeled_epoch += 1
15            unlabeled_trainloader.sampler.set_epoch(unlabeled_epoch)
16            unlabeled_iter = iter(unlabeled_trainloader)
17            (inputs_u_w, inputs_u_s), _ = unlabeled_iter.next()

```

得到强扩增样本和弱扩增样本, 分别为 logits_u_s 和 logits_u_w:

```

1 logits = model(inputs)
2 logits = de_interleave(logits, 2*args.mu+1)
3 logits_x = logits[:batch_size]
4 logits_u_w, logits_u_s = logits[batch_size:].chunk(2)

```

对有标记样本计算交叉熵：

```
1 Lx = F.cross_entropy(logits_x, targets_x, reduction='mean')
```

使用 `weak_augment` 样本进行计算，得到伪标签 `pseudo label` 和 `mask`。其中，`mask` 的作用是用来筛选出那些具有最大预测概率超过阈值的样本，这些样本可以被使用；而那些预测概率未达到阈值的样本则被排除在外，不能被使用：

```
1 pseudo_label = torch.softmax(logits_u_w.detach()/args.T, dim=-1)
2 max_probs, targets_u = torch.max(pseudo_label, dim=-1)
3 mask = max_probs.ge(args.threshold).float()
```

计算无标记样本的损失函数 L_u ，并通过 `mask` 进行样本筛选：

```
1 Lu = (F.cross_entropy(logits_u_s, targets_u, reduction='none') * mask).mean()
```

计算完整的损失函数，然后反向传播训练即可：

```
1 loss = Lx + args.lambda_u * Lu
```

3.2 扩散生成模型

扩散模型的核心就在于训练噪音预测模型，由于噪音和原始数据是同维度的，我们可以选择采用 AutoEncoder 架构来作为噪音预测模型。DDPM 所采用的模型是一个基于 residual block 和 attention block 的 U-Net 模型。

U-Net 的核心模块是 residual block，它包含两个卷积层和批归一化层。卷积层使用 3×3 的卷积核和步幅为 1 的卷积操作，激活函数使用 GELU。如果 `is_res` 为 True 且输入和输出的通道数相同，则将残差添加到输入上：

```
1 class residual_conv_block(nn.Module):
2     def __init__(
3         self, input_channels: int, output_channels: int, res_flag: bool = False
4     ) -> None:
5         super().__init__()
6         self.same_channels = input_channels == output_channels
7         self.res_flag = res_flag
8         self.conv1 = nn.Sequential(
9             nn.Conv2d(input_channels, output_channels, 3, 1, 1),
10            nn.BatchNorm2d(output_channels),
11            nn.GELU(),
12        )
13        self.conv2 = nn.Sequential(
14            nn.Conv2d(output_channels, output_channels, 3, 1, 1),
15            nn.BatchNorm2d(output_channels),
16            nn.GELU(),
17        )
18        def forward(self, x: torch.Tensor) -> torch.Tensor:
19            if self.res_flag:
20                x1 = self.conv1(x)
21                x2 = self.conv2(x1)
22                # 如果通道数相同，则将残差添加到输入上
23                if self.same_channels:
24                    out = x + x2
25                else:
26                    out = x1 + x2
27                return out / 1.414 # 归一化
28            else:
```

```

29         x1 = self.conv1(x)
30         x2 = self.conv2(x1)
31         return x2

```

对于上采样模块和下采样模块,其分别可以采用插值和 stride=2 的 conv 或者 pooling 来实现。在上采样模块中,我使用 ResidualConvBlock 类作为上采样模块的主要组件,使用 nn.ConvTranspose2d 进行上采样操作。在下采样模块中使用 ResidualConvBlock 类作为下采样模块的主要组件,使用 nn.MaxPool2d 进行下采样操作,具体代码如下:

```

1  #处理并下采样图像特征图
2  class Downsample(nn.Module):
3      def __init__(self, input_channels, output_channels):
4          super(Downsample, self).__init__()
5          layers = [residual_conv_block(input_channels, output_channels), nn.MaxPool2d(2)]
6          self.model = nn.Sequential(*layers)
7      def forward(self, x):
8          return self.model(x)
9  #处理并上采样图像特征图
10 class Upsample(nn.Module):
11     def __init__(self, input_channels, output_channels):
12         super(Upsample, self).__init__()
13         layers = [
14             nn.ConvTranspose2d(input_channels, output_channels, 2, 2),
15             residual_conv_block(output_channels, output_channels),
16             residual_conv_block(output_channels, output_channels),
17         ]
18         self.model = nn.Sequential(*layers)
19     def forward(self, x, skip):
20         x = torch.cat((x, skip), 1) # 在通道维度上拼接张量
21         x = self.model(x)
22         return x

```

定义 Embedding_FC 类,该类是一个通用的单层全连接神经网络,用于实现 embedding:

```

1  class Embedding_FC(nn.Module):
2      def __init__(self, input_dim, emb_dim):
3          super(Embedding_FC, self).__init__()
4          self.input_dim = input_dim
5          layers = [
6              nn.Linear(input_dim, emb_dim),
7              nn.GELU(),
8              nn.Linear(emb_dim, emb_dim),
9          ]
10         self.model = nn.Sequential(*layers)
11     def forward(self, x):
12         x = x.view(-1, self.input_dim)
13         return self.model(x)

```

上面实现了 U-Net 的所有组件,接下来将其进行组合来实现 U-Net, U-Net 模型是 DDPM 模型的主要组件。包含初始卷积层、下采样模块、平均池化层、时间嵌入层、上下文嵌入层和上采样模块等,具体实现如下:

```

1  class U_Net(nn.Module):
2      def __init__(self, input_channels, n_feat=256, n_classes=10):
3          super(U_Net, self).__init__()
4          self.input_channels = input_channels
5          self.n_feat = n_feat
6          self.n_classes = n_classes

```

```

7 self.init_conv = residual_conv_block(input_channels, n_feat, res_flag=True) # 初始卷积层
8 self.down1 = Downsample(n_feat, n_feat) # 下采样模块1
9 self.down2 = Downsample(n_feat, 2 * n_feat) # 下采样模块2
10 self.to_vec = nn.Sequential(nn.AvgPool2d(7), nn.GELU()) # 平均池化和GELU激活函数
11 self.timeembed1 = Embedding_FC(1, 2 * n_feat) # time embedding1
12 self.timeembed2 = Embedding_FC(1, 1 * n_feat) # time embedding2
13 self.contextembed1 = Embedding_FC(n_classes, 2 * n_feat) # 上下文嵌入层1
14 self.contextembed2 = Embedding_FC(n_classes, 1 * n_feat) # 上下文嵌入层2
15 self.up0 = nn.Sequential(
16     nn.ConvTranspose2d(2 * n_feat, 2 * n_feat, 7, 7), # 转置卷积进行上采样
17     nn.GroupNorm(8, 2 * n_feat), # 组归一化
18     nn.ReLU(),
19 )
20 self.up1 = Upsample(4 * n_feat, n_feat) # 上采样模块1
21 self.up2 = Upsample(2 * n_feat, n_feat) # 上采样模块2
22 self.out = nn.Sequential(
23     nn.Conv2d(2 * n_feat, n_feat, 3, 1, 1), # 卷积层
24     nn.GroupNorm(8, n_feat), # 组归一化
25     nn.ReLU(),
26     nn.Conv2d(n_feat, self.input_channels, 3, 1, 1), # 最终的卷积层
27 )
28 def forward(self, x, c, t, context_mask):
29     # x是(带噪声的)图像, c是上下文标签, t是时间步
30     # context_mask表示哪些样本要屏蔽上下文
31     x = self.init_conv(x) # 初始卷积处理
32     down1 = self.down1(x) # 下采样1
33     down2 = self.down2(down1) # 下采样2
34     hiddenvec = self.to_vec(down2) # 将特征图转换为向量表示
35     # 将上下文标签c转换为one-hot
36     c = nn.functional.one_hot(c, num_classes=self.n_classes).type(torch.float)
37     # 根据context_mask屏蔽上下文
38     context_mask = context_mask[:, None]
39     context_mask = context_mask.repeat(1, self.n_classes)
40     context_mask = (-1 * (1 - context_mask)) # 需要翻转0和1
41     c = c * context_mask
42     # 嵌入上下文和时间步骤
43     cemb1 = self.contextembed1(c).view(-1, self.n_feat * 2, 1, 1) # 上下文嵌入层1
44     temb1 = self.timeembed1(t).view(-1, self.n_feat * 2, 1, 1) # time embedding1
45     cemb2 = self.contextembed2(c).view(-1, self.n_feat, 1, 1) # 上下文嵌入层2
46     temb2 = self.timeembed2(t).view(-1, self.n_feat, 1, 1) # time embedding2
47     up1 = self.up0(hiddenvec) # 上采样0
48     up2 = self.up1(cemb1 * up1 + temb1, down2) # 上采样1
49     up3 = self.up2(cemb2 * up2 + temb2, down1) # 上采样2
50     out = self.out(torch.cat((up3, x), 1)) # 将上采样的结果与初始输入拼接, 生成最终的输出
51     return out

```

对于扩散过程，其主要的参数就是 timesteps 和 noise schedule，DDPM 采用范围为 [0.0001, 0.02] 的线性 noise schedule，其默认采用的总扩散步数为 1000。定义 get_schedules() 函数返回 DDPM 采样和训练过程的预先计算的调度表：

```

1 #返回DDPM采样和训练过程的预先计算的调度表。
2 def get_schedules(beta1, beta2, T):
3     assert beta1 < beta2 < 1.0 #beta1,beta2在(0, 1)"
4     # 每个时间步的 beta_t
5     beta_t = (beta2 - beta1) * torch.arange(0, T + 1, dtype=torch.float32) / T + beta1
6     sqrt_beta_t = torch.sqrt(beta_t)
7     # 每个时间步的 alpha_t
8     alpha_t = 1 - beta_t
9     log_alpha_t = torch.log(alpha_t)

```

```

10 # 计算 alphas_bar_t
11 alphas_bar_t = torch.cumsum(log_alpha_t, dim=0).exp()
12 # sqrt_alpha_bar_t
13 sqrt_alpha_bar_t = torch.sqrt(alphas_bar_t)
14 # oneover_sqrt_alpha_bar_t
15 oneover_sqrt_alpha_bar_t = 1 / torch.sqrt(alphas_bar_t)
16 # sqrt_beta_t
17 sqrt_beta_t = torch.sqrt(1 - alphas_bar_t)
18 # mab_over_sqrt_beta_t, 即 (1 - alpha_t) / sqrt_beta_t
19 mab_over_sqrt_beta_t_inv = (1 - alpha_t) / sqrt_beta_t
20 return {
21     "alpha_t": alpha_t,
22     "oneover_sqrt_alpha_bar_t": oneover_sqrt_alpha_bar_t,
23     "sqrt_beta_t": sqrt_beta_t,
24     "alphas_bar_t": alphas_bar_t,
25     "sqrt_alpha_bar_t": sqrt_alpha_bar_t,
26     "sqrt_beta_t": sqrt_beta_t,
27     "mab_over_sqrt_beta_t_inv": mab_over_sqrt_beta_t_inv,
28 }

```

定义 DDPM 类如下，forward() 为前向传播方法，用于训练模型。参数 x 是输入数据， c 是上下文标签。在训练过程中，随机选择样本时间步 t 和噪声 $noise$ 。然后计算 x_t ，将输入数据 x 与噪声 $noise$ 相加，其中使用了一些缓冲区的计算结果。接着按照一定概率进行上下文 dropout，生成 context_mask。最后返回噪声与通过神经网络模型预测的噪声之间的均方误差。sample() 为采样方法，用于生成数据。首先初始化初始噪声 x_i 和上下文标签 c_i 。然后扩展批次并生成上下文掩码 context_mask。接着循环采样时间步，逐步生成样本。最终返回生成的样本 x_i 和保存的生成步骤 x_i_store 。具体代码如下：

```

1 class DDPM(nn.Module):
2     def __init__(self, nn_model, betas, n_T, device, drop_chance=0.1):
3         super(DDPM, self).__init__()
4         self.nn_model = nn_model.to(device)
5         # 使用 get_schedules 计算调度表
6         for k, v in get_schedules(betas[0], betas[1], n_T).items():
7             self.register_buffer(k, v)
8         self.n_T = n_T
9         self.device = device
10        self.drop_chance = drop_chance
11        self.LOSS_MSE = nn.MSELoss()
12
13    def forward(self, x, c):
14        # 训练，随机选择样本 t 和噪声
15        _ts = torch.randint(1, self.n_T+1, (x.shape[0],)).to(self.device) # t ~ Uniform(0, n_T)
16        noise = torch.randn_like(x) # eps ~ N(0, 1)
17        x_t = (
18            self.sqrt_alpha_bar_t[_ts, None, None, None] * x
19            + self.sqrt_beta_t[_ts, None, None, None] * noise
20        )
21        # 以一定概率进行 dropout
22        context_mask = torch.bernoulli(torch.zeros_like(c) + self.drop_chance).to(self.device)
23        # 返回加入噪声与预测的噪声之间的 mse
24        return self.LOSS_MSE(noise, self.nn_model(x_t, c, _ts / self.n_T, context_mask))
25
26    def sample(self, n_sample, size, device, guide_w=0.0):
27        x_i = torch.randn(n_sample, *size).to(device) # x_T ~ N(0, 1), 采样初始噪声
28        c_i = torch.arange(0, 10).to(device) # 上下文标签仅循环遍历 mnist 标签
29        c_i = c_i.repeat(int(n_sample / c_i.shape[0]))
30        # 在测试时不进行上下文 dropout

```

```

30 context_mask = torch.zeros_like(c_i).to(device)
31 c_i = c_i.repeat(2)
32 context_mask = context_mask.repeat(2)
33 context_mask[n_sample:] = 1.
34 x_i_store = [] # 保留生成步骤以便绘图
35 print()
36 for i in range(self.n_T, 0, -1):
37     print(f'sampling timestep {i}', end='\r')
38     t_is = torch.tensor([i / self.n_T]).to(device)
39     t_is = t_is.repeat(n_sample, 1, 1, 1)
40     x_i = x_i.repeat(2, 1, 1, 1)
41     t_is = t_is.repeat(2, 1, 1, 1)
42     z = torch.randn(n_sample, *size).to(device) if i > 1 else 0
43     eps = self.nn_model(x_i, c_i, t_is, context_mask)
44     eps1 = eps[:n_sample]
45     eps2 = eps[n_sample:]
46     eps = (1 + guide_w) * eps1 - guide_w * eps2
47     x_i = x_i[:n_sample]
48     x_i = (
49         self.oneover_sqrt_alpha_bar[i] * (x_i - eps * self.mab_over_sqrt_alpha_bar[i])
50         + self.sqrt_beta_bar_t[i] * z
51     )
52     if i % 20 == 0 or i == self.n_T or i < 8:
53         x_i_store.append(x_i.detach().cpu().numpy())
54 x_i_store = np.array(x_i_store)
55 return x_i, x_i_store

```

用于训练和测试的代码过长，就不在此赘述，详细代码见 ddpm.py 文件。

4 实验结果

4.1 半监督图像分类

4.1.1 图像分类效果

- 使用 MixMatch 算法，以 WideResNet-28-2 作为 Backbone 网络，深度为 28，扩展因子为 2，在 CIFAR-10 数据集上进行半监督图像分类实验，在分别使用 40,250,4000 张标注数据的情况下的损失函数变化图分别如图 3、图 4、图 5 所示。训练过程中在验证集和测试集上的分类正确率变化分别如图 6 中的 (a)、(b)、(c) 所示。

可以看到使用的标注数据越多，损失函数下降的就越快，MixMatch 算法收敛的也就越快。使用 250 张标注数据和 4000 张标注数据时，算法在很短的时间内就能收敛，而使用 40 张标注数据时，过了 300 个 epoch 后还没有明显的收敛趋势。

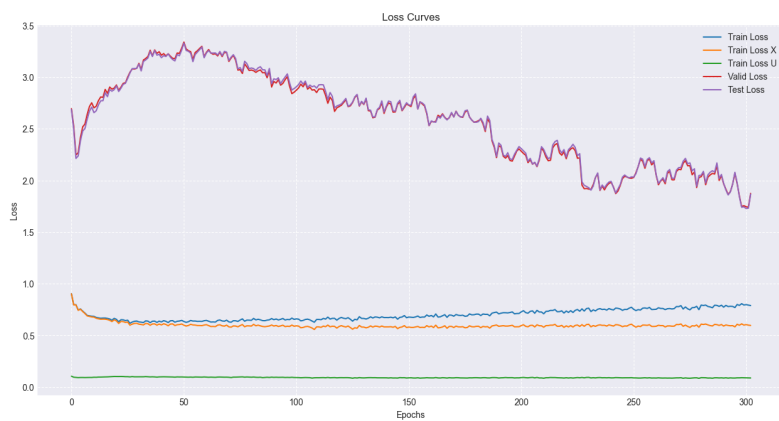


图 3: MixMatch 使用 40 张标注数据的情况下的损失函数变化图

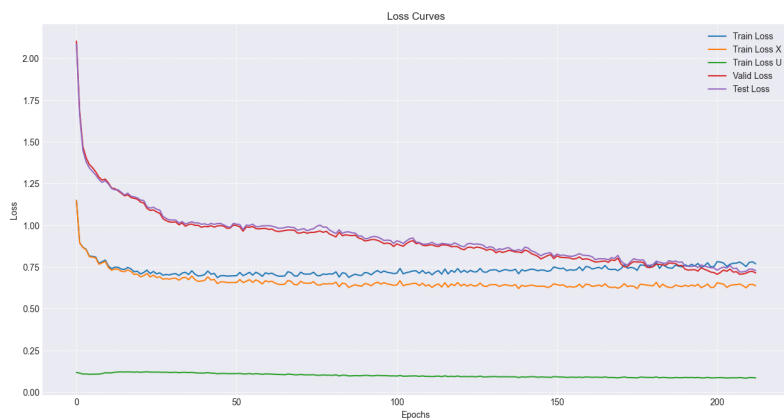


图 4: MixMatch 使用 250 张标注数据的情况下的损失函数变化图

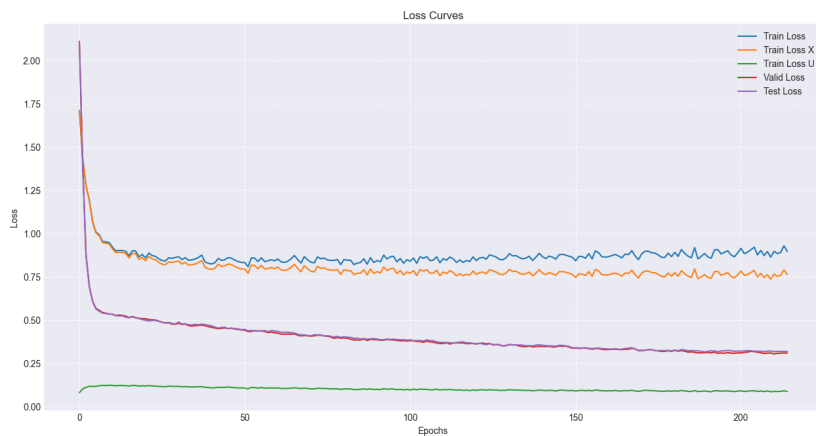
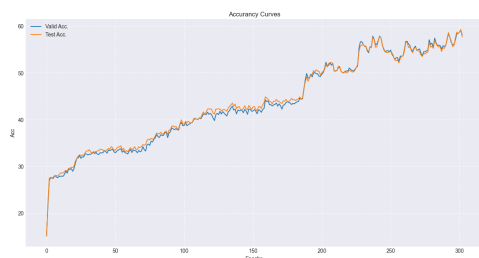
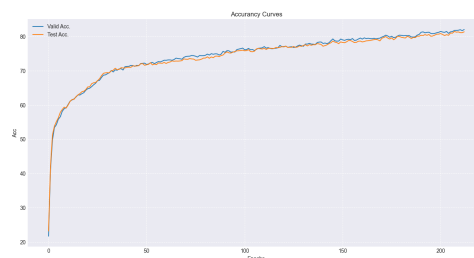


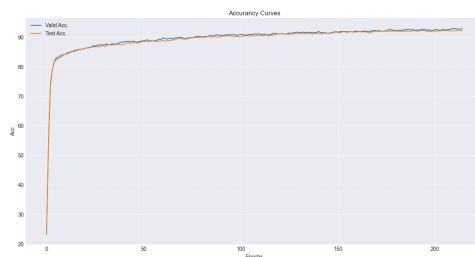
图 5: MixMatch 使用 4000 张标注数据的情况下的损失函数变化图



(a) 使用 40 张标注数据



(b) 使用 250 张标注数据



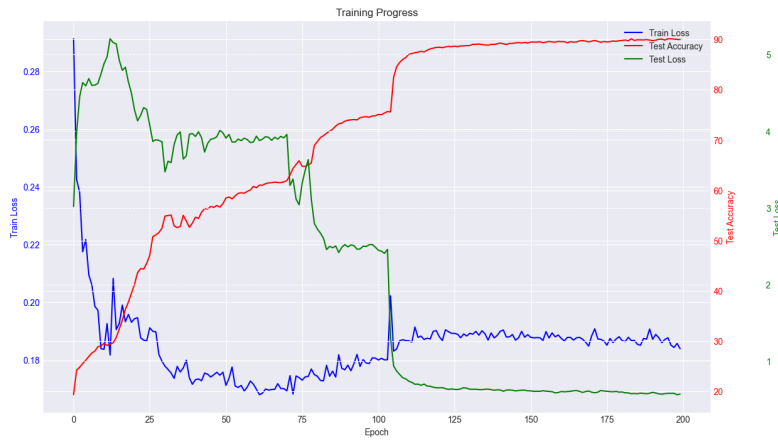
(c) 使用 4000 张标注数据

图 6: MixMatch 使用不同数量的标注数据情况下的正确率变化图

可以看到，在分类正确率的表现上，标注数据数量越多，模型的表现要越好。使用 40 张标注数据时，即使训练了 300 个 epoch，在测试集和验证集上的正确率也只有 60%，而使用 250 张标注数据时，训练了 200 个 epoch 后正确率就能达到 80%，使用 4000 张标注数据时，不用 100 个 epoch 模型的正确率就突破了 90%，符合预期。

- 使用 FixMatch 算法，以 WideResNet-28-2 作为 Backbone 网络，深度为 28，扩展因子为 2，在 CIFAR-10 数据集上进行半监督图像分类实验，在分别使用 40,250,4000 张标注数据的情况下的，在训练集和测试集上的损失函数、在测试集上的准确率变化图分别如图 7 所示。

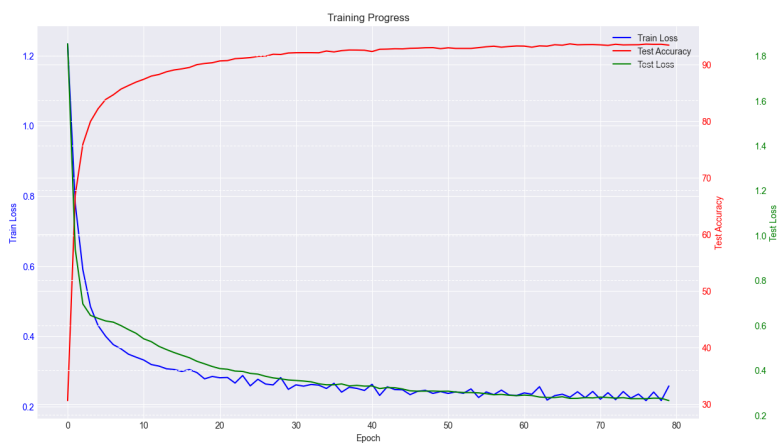
在 FixMatch 算法的训练过程中，使用的标注数据越多，损失函数下降的就越快，MixMatch 算法收敛的也就越快。在分类正确率的表现上也如此，标注数据数量越多，模型的表现要越好。最终模型集上的正确率都可以突破 90%，但使用 40 张标注数据时需要花费接近 200 个 epoch，使用 250 张标注数据时需要花费 50 个 epoch，而使用 4000 张标注数据时只需要 20 个 epoch 便可以达到 90% 以上的正确率。



(a) 使用 40 张标注数据



(b) 使用 250 张标注数据



(c) 使用 4000 张标注数据

图 7: FixMatch 使用不同数量的标注数据情况下 train_loss、test_acc 和 test_loss 变化图，不同颜色的纵坐标对应不同的数据指标。

4.1.2 和 TorchSSL 的对比

使用 TorchSSL (<https://github.com/TorchSSL/TorchSSL>) 中提供的 MixMatch 和 FixMatch 的实现进行半监督训练和测试，对比自己实现的算法和 TorchSSL 中的实现的效果如表 1 所示。

	40	250	4000
TorchSSL MixMatch	63.81±6.48	86.37±0.59	93.34±0.26
TorchSSL FixMatch	92.53±0.28	95.14±0.05	95.79±0.08
My MixMatch	58.63	81.36	92.27
My FixMatch	90.14	92.98	94.47

表 1: 和 TorchSSL 的对比，表项表示最终在测试集上的分类正确率 (%)

4.1.3 MixMatch 和 FixMatch 的相同点和不同点

相同点：

- MixMatch 和 FixMatch 都属于半监督学习方法，通过同时利用有标签数据和无标签数据进行模型训练，以提高模型性能。
- 两种方法都使用了数据增强技术，通过对有标签和无标签数据应用各种变换和扰动，以扩充训练数据集，增加模型的鲁棒性和泛化能力。

不同点：

- 在 FixMatch 中，标签数据被用于计算损失函数，但无标签数据只用于生成伪标签，而不直接参与损失计算。而在 MixMatch 中，有标签和无标签数据都被用于计算损失函数，并且有标签数据的权重会更高。
- 在 FixMatch 中，使用了一个阈值来选择生成的伪标签，只有当模型对无标签样本的预测置信度高于阈值时，才会使用该样本的伪标签。而在 MixMatch 中，对于每个无标签样本，使用模型预测的多个伪标签，并将它们与有标签数据一起进行交叉熵损失的计算。
- 在 FixMatch 中，对于每个无标签样本，仅使用生成的伪标签进行损失计算，而不对预测输出进行进一步的利用。而在 MixMatch 中，对于每个无标签样本，使用模型预测输出的概率分布，并将其与有标签数据一起进行损失计算。

4.2 扩散生成模型

运行代码，一开始的生成效果不是很好，在经过大约 20 个 epoch 的训练后，模型的生成能力已经非常理想，下图是分别在 MNIST 数据集和 Fashion MNIST 数据集上的生成效果图，图片上方一列表示生成的图像，下方表示原图像。

动图形式的生成效果图见 result 文件夹。

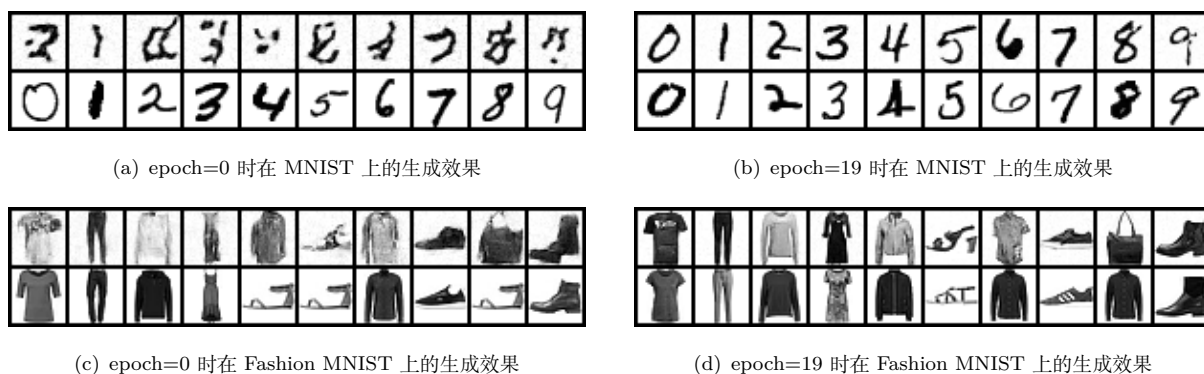


图 8: 不同迭代次数, 两个数据集上的生成的图像和原图像的对比

5 实验总结

5.1 遇到的困难

在完成本次作业的过程中, 阅读原始论文和相关参考资料需要一定的时间和精力, 理解深度学习算法的原理和细节也有一定难度。实现 MixMatch 和 FixMatch 算法需要熟悉 PyTorch 框架, 并且需要正确地设置各种模型和参数。这对我来说是一个挑战, 但好在有大量的资料可以参考, 能让我顺利地完成任务。

5.2 实验感想

实验过程中的困难和挑战有助于深化对深度学习算法的理解。通过阅读论文、实现算法和调参, 我对 MixMatch 和 FixMatch 的工作原理和特点有了更深入的了解。除此之外, 通过与 TorchSSL 中提供的实现进行对比, 我能够评估自己实现的算法与开源实现之间的差异。这有助于我了解优秀开源工具的实现细节和性能。

此外, 通过实现简单版的 DDPM, 并在 MNIST 和 Fashion MNIST 数据集上进行训练和测试, 我对扩散生成模型的原理和实现印象更加深刻了, 这让我受益匪浅。

参考文献

- [BCG⁺19] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin A Raffel. Mixmatch: A holistic approach to semi-supervised learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [SBC⁺20] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 596–608. Curran Associates, Inc., 2020.

- [YZS⁺22] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Yingxia Shao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models: A comprehensive survey of methods and applications. *arXiv preprint arXiv:2209.00796*, 2022.